

Microservizi Con SpringCloud

Introduzione Microservizi

Definizione. Un microservizio è un'applicazione piccola, autonoma e indipendentemente distribuibile, responsabile di **una sola area funzionale** e proprietaria esclusiva dei propri dati.

Il docente lo spiega per contrasto con il **monolite**: un'unica applicazione che contiene interfaccia grafica, business logic e accesso al database, impacchettata in un **.war** e **depositata dentro un application server** (nella trascrizione "Wall the fly" è WildFly).

Con i microservizi si ribalta la situazione su due punti:

1. **Dati separati**: ogni microservizio ha il suo database/tabella ed è l'unico responsabile di quella porzione di dati.
2. **Server invertito**: il server non contiene più l'applicazione, è l'applicazione che contiene il server. Spring Boot embedda Tomcat dentro il jar. Se hai 4 microservizi, hai 4 Tomcat che partono, uno per jar.

Pro: scalabilità. Se un microservizio è sovraccarico ne avvii più istanze *solo di quello*, senza duplicare l'intero sistema.

Contro: complessità di gestione. Il docente elenca esattamente i problemi che le lezioni successive risolveranno uno a uno:

Problema	Soluzione (lezione)
Dove finiscono i file di configurazione?	Spring Cloud Config
Come chiamo un servizio da un altro?	OpenFeign
Come faccio a sapere dove sta l'altro servizio?	Eureka (registro)
Come distribuisco il carico su più istanze?	Spring Cloud LoadBalancer
Come faccio un punto di ingresso unico?	Spring Cloud Gateway
Come isolo un servizio guasto senza bloccare tutto?	Resilience4j

Il criterio di scelta che dà: se conosci già la tua utenza, resta monolitico, non aggiungere complessità inutile. Se hai un'applicazione internet con crescita imprevedibile, vai a microservizi.

Il codice

Il microservizio di base è `HelloWorldSpringBoot`, creato con *New → Spring Starter Project* e due dipendenze: **Spring Web** (per esporre REST) e **DevTools** (riavvio automatico a ogni modifica).

```
@RestController
```

```
@CrossOrigin(origins = "http://localhost:4200")
```

```
@RequestMapping("/api")
```

```

public class HelloController {

    @Value("${messaggio}")
    private String tmp;

    @RequestMapping(value = "/ciao", method = RequestMethod.GET, produces =
"application/json")
    public ResponseEntity<String> helloWorld() {
        System.out.println("tmp:" + tmp);
        return new ResponseEntity<String>("'"ciao mamma'", HttpStatus.OK);
    }

    @GetMapping(value = "/elencoArticoli")
    public ResponseEntity<List<Articoli>> elencoArticoli() { ... }
}

```

e application.properties:

```

server.port=8080
spring.application.name=HelloWorldSpringBoot
messaggio = Ciao Mondo

```

Due cose da fissare qui perché tornano in **tutte** le lezioni successive:

- **server.port**: in un ecosistema di microservizi devi settarla sempre, altrimenti tutti partono su 8080 e vanno in conflitto.
- **spring.application.name**: è il **nome logico** del servizio. È la chiave con cui Config Server, Eureka, Feign e Gateway lo identificheranno. Il docente insiste: "chiunque ci vorrà chiamare dovrà utilizzare il nostro nome".

Nel progetto ci sono anche `application-prod.properties` e `application-test.properties` (con `listino = Inverno` / `listino = Autunno`): sono i **profili** Spring, l'anticipazione del problema che Config Server risolverà in modo centralizzato.

Introduzione a Spring Cloud

Definizione. Spring Cloud non è un singolo framework: è un **insieme di moduli** che si aggiungono a Spring Boot per risolvere i problemi ricorrenti dei sistemi distribuiti (configurazione, service discovery, routing, load balancing, resilienza, tracing).

Come si usa: si aggiunge il **BOM** (Bill of Materials) di Spring Cloud al `pom.xml`, che allinea le versioni di tutti i moduli, e poi si aggiunge solo la starter del modulo che serve.

Dal `pom.xml` reale dei progetti (`HelloWorldSpringCloudEurekaClient-3`):

```
<properties>
  <java.version>17</java.version>
  <spring-cloud.version>2022.0.3</spring-cloud.version>
</properties>

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>${spring-cloud.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Le versioni di Spring Cloud hanno **due nomenclature** e nei tuoi progetti trovi entrambe:

- **Nomi di stazioni della metro di Londra:** `Hoxton.M3`, `Hoxton.SR5` (progetti vecchi, con Spring Boot 2.2/2.3)
- **Anno.release:** `2022.0.3` (progetti nuovi, con Spring Boot 3.1)

Non è un dettaglio estetico: `2022.0.x` richiede Spring Boot 3.x e Java 17, e ha cambiato alcune cose (lo vedrai tra un attimo con `bootstrap.yml`).

Spring Cloud Config – Introduzione ed esempio

Definizione. Spring Cloud Config è il modulo che **esternalizza e centralizza la configurazione**. Invece di avere 200 `application.properties` sparsi in 200 microservizi, hai un unico **Config Server** che legge i file da un repository (file system o Git) e li serve via HTTP ai **Config Client**.

Il problema che risolve, con le parole del docente: "immaginate un progetto composto da 200 microservizi. Io dovrei configurarli tutti e 200, se poi cambia qualcosa li dovrei riavviare. È un mezzo casino."

Architettura in tre pezzi:

```
[ Repository ] ← [ Config Server :8888 ] ← [ Client 1, Client 2, ... N ]  
(file o Git)      legge i file          chiedono la config all'avvio
```

Il Config Server (HelloWorldSpringCloudConfig)

Dipendenza: spring-cloud-config-server. Poi **una sola annotazione**:

```
@SpringBootApplication  
@EnableConfigServer      // ← questo trasforma l'app in un Config Server  
public class HelloWorldSpringCloudConfigApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(HelloWorldSpringCloudConfigApplication.class, args);  
    }  
}
```

La configurazione è in bootstrap.yml, e qui la sorgente è il **file system** (profilo native):

```
server:  
  port: 8888  
  
spring:  
  cloud:  
    config:  
      server:  
        native:  
          search-locations:  
            file:///C:/Prova/  
  profiles:  
    active:  
      - native
```

Dentro C:/Prova/ c'è un application.properties con dentro welcome.message.

Nota tecnica importante. Questo progetto usa bootstrap.yml, che funzionava con Spring Cloud Hoxton. Da Spring Cloud 2020.0 in poi il bootstrap context è **disabilitato di default**: o aggiungi spring-cloud-starter-bootstrap, oppure (modo moderno) usi
spring.config.import=configserver:http://localhost:8888

dentro il normale `application.yml`. Nei progetti più recenti dello zip (Config-2) infatti il docente usa `application.properties` normale.

La porta **8888** è la convenzione de facto per il Config Server, esattamente come 8761 lo è per Eureka.

Il Config Client (`HelloWorldSpringCloudConfigClient`)

Dipendenza: `spring-cloud-starter-config`. Configurazione:

```
server:
  port: 5071

spring:
  application:
    name: HelloWorldSpringCloudConfigClientApplication
  cloud:
    config:
      uri: http://localhost:8888    # ← dove sta il Config Server
```

E il controller che consuma la property remota:

```
@RestController
@RequestMapping("/api")
public class HelloController {

    @Value("${welcome.message}") // ← NON è nel properties locale!
    private String welcomeMessage;

    @RequestMapping(value = "/ciao", method = RequestMethod.GET, produces =
"application/json")
    public ResponseEntity<String> helloWorld() {
        return new ResponseEntity<String>(welcomeMessage, HttpStatus.OK);
    }
}
```

Il punto chiave da capire: `welcome.message` non esiste da nessuna parte dentro il progetto client. All'avvio il client contatta `localhost:8888`, chiede la configurazione **per il nome della propria applicazione**, la riceve e solo allora popola il campo con `@Value`. Se chiami `/api/ciao` ti risponde "benvenuto", che è il valore che sta nel file su `C:/Prova/`.

L'endpoint di test

Il Config Server espone la configurazione come API REST. Il docente lo testa con Postman:

```
http://localhost:8888/{applicazione}/{profilo}/{label}
```

esempio: `http://localhost:8888/config-client/default/master`

Questo è il modo per verificare che il server funzioni **prima ancora** di scrivere il client.

Spring Cloud Config – Esempio con Git

Stessa cosa della lezione precedente, ma con il repository che è **Git**, che il docente definisce "la situazione più comune" (e lo è: ti dà versioning, storico e rollback della configurazione).

Preparazione del repository

Il docente crea una cartella `C:\Prova\config-server-repo` e ci mette dentro tre file, poi:

```
git init
```

```
git add .
```

```
git commit -m "commento"
```

La convenzione di naming dei file è la parte più importante della lezione:

```
{nome-applicazione}-{profilo}.properties
```

Quindi nel repo ci sono:

File	A chi serve
<code>application.properties</code>	a tutte le applicazioni (config comune)
<code>HelloWorldSpringCloudConfigClient-2-dev.properties</code>	solo al client -2, profilo dev
<code>HelloWorldSpringCloudConfigClient-2-prod.properties</code>	solo al client -2, profilo prod

Il Config Server con Git (HelloWorldSpringCloudConfig-2)

```
@SpringBootApplication
```

```
@EnableConfigServer
```

```
public class HelloWorldSpringCloudConfig2Application { ... }
```

```
#Server port
```

```
server.port = 8888
```

```
#Git repo location
```

```
spring.cloud.config.server.git.uri=C:\\Prova\\config-server-repo
```

```
#Verify any repository issue in service startup
```

```
spring.cloud.config.server.git.cloneOnStart=true
```

```
#Disable security of the Management endpoint
```

```
management.security.enabled=false
```

Riga per riga:

- **git.uri**: qui è un percorso locale, ma in produzione ci metti `https://github.com/azienda/config-repo`.
- **cloneOnStart=true**: clona il repository all'avvio invece che alla prima richiesta. Serve a **fallire subito** se il repository non è raggiungibile, invece di scoprirlo a runtime.
- **management.security.enabled=false**: solo per semplificare la demo.

Il test

```
http://localhost:8888/HelloWorldSpringCloudConfigClient-2/prod
```

→ restituisce il contenuto di ...-prod.properties

```
http://localhost:8888/HelloWorldSpringCloudConfigClient-2/dev
```

→ restituisce il contenuto di ...-dev.properties

Cambiando solo l'ultima parte dell'URL ottieni configurazioni diverse. **Il client sceglie quale riceve** dichiarando il proprio profilo attivo:

```
spring.application.name=HelloWorldSpringCloudConfigClient-2
```

```
spring.profiles.active=dev
```

```
spring.cloud.config.uri=http://localhost:8888
```

Con **active=dev** il client riceve il messaggio del file dev. Cambi una riga e vai in produzione, senza toccare il codice.

OpenFeign

Definizione. OpenFeign è un **client HTTP dichiarativo**: descrivi il servizio remoto con un'interfaccia Java annotata, e Feign genera a runtime l'implementazione che fa le chiamate HTTP. Tu chiami un metodo Java normale, dietro le quinte parte una richiesta REST.

Il problema: in un'architettura a microservizi capita continuamente che un servizio, per completare il suo lavoro, debba invocare un altro. Si può fare in tanti modi (`RestTemplate`, `WebClient`), Feign è il più pulito.

Il servizio chiamato

È HelloWorldSpringBoot della lezione 1, su porta 8080, che espone tre endpoint:

URL	Ritorna
/api/ciao	Una String
/api/getArticolo	Un oggetto Articoli
/api/elencoArticoli	Una Lista<Articoli>

Il servizio chiamante (HelloWorldSpringBootOpenFeign, porta 8081)

Passo 1 — dipendenza: spring-cloud-starter-openfeign.

Passo 2 — abilitare Feign sulla classe principale:

```
@SpringBootApplication
@EnableFeignClients()    // ← fa scansionare le interfacce @FeignClient
public class HelloWorldSpringBootOpenFeignApplication { ... }
```

Passo 3 — dichiarare l'interfaccia. Questa è la parte centrale:

```
@FeignClient(name = "HelloWorldSpringBoot", url = "localhost:8080")
public interface TestClient {

    @RequestMapping(value = "/api/ciao", method = RequestMethod.GET, produces =
"application/json")
    public ResponseEntity<String> helloWorld();

    @RequestMapping(value = "/api/getArticolo", method = RequestMethod.GET, produces =
"application/json")
    public ResponseEntity<Articoli> getArticolo();

    @GetMapping(value = "/api/elencoArticoli")
    public ResponseEntity<List<Articoli>> elencoArticoli();
}
```

Osservazioni fondamentali:

- **Non c'è nessuna implementazione.** Solo un'interfaccia. Feign genera il proxy a runtime.
- Il metodo pratico che suggerisce il docente: **copia-incolla la firma del metodo e la sua annotazione** dal controller del servizio remoto dentro l'interfaccia. Il mapping deve corrispondere.

- **url="localhost:8080"** è l'indirizzo *cablato nel codice*. Il docente lo fa qui perché non c'è ancora Eureka. Nella lezione su Eureka questo url **sparirà** e resterà solo il name.
- La classe **Articoli** va **ridichiarata** anche nel progetto chiamante, perché la deserializzazione JSON ha bisogno del tipo.

Passo 4 — usarla come un normale bean Spring:

```
@RestController
@RequestMapping("/provaws")
public class ProvaController {

    @Autowired
    private TestClient testClient;    // ← Spring inietta il proxy Feign

    @RequestMapping(value = "/helloWorld", method = RequestMethod.GET, produces =
"application/json")
    public ResponseEntity<String> helloWorld() {
        return testClient.helloWorld();    // ← sembra una chiamata locale, è HTTP
    }

    @RequestMapping(value = "/testArticolo", method = RequestMethod.GET, produces =
"application/json")
    public ResponseEntity<String> testArticolo() {
        Articoli a = testClient.getArticolo().getBody();
        return new ResponseEntity<String>(a.getDescrizione(), HttpStatus.OK);
    }

    @RequestMapping(value = "/testLista", method = RequestMethod.GET, produces =
"application/json")
    public ResponseEntity<List<Articoli>> elencoArticoli() {
        List<Articoli> a = testClient.elencoArticoli().getBody();
        return new ResponseEntity<List<Articoli>>(a, HttpStatus.OK);
    }
}
```

Il flusso completo

Browser → :8081/provaws/helloWorld → [proxy Feign] → :8080/api/ciao → "ciao mamma"

Il tuo codice non vede mai la chiamata HTTP: è tutto nascosto dietro `testClient.helloWorld()`.

Avviare più istanze

Definizione. Istanziare più copie identiche dello stesso microservizio, su porte diverse, per distribuire il carico. È il presupposto tecnico di tutte le lezioni sul load balancing.

Il problema: se un microservizio è un collo di bottiglia, ne vuoi 3-4 copie. Ma non puoi avviarle sulla stessa porta ("la porta è già in uso").

Soluzione: la porta **non** si scrive nel `properties`, si passa come parametro di JVM all'avvio, sovrascrivendo il valore del file.

Metodo 1 — riga di comando

```
cd workspace\HelloWorldSpringBoot\target
```

```
java -Dserver.port=8081 -jar HelloWorldSpringBoot-0.0.1-SNAPSHOT.jar
```

e in un secondo prompt:

```
java -Dserver.port=8082 -jar HelloWorldSpringBoot-0.0.1-SNAPSHOT.jar
```

Il `-D` definisce una **system property** della JVM, che in Spring Boot ha priorità più alta rispetto ad `application.properties`. Per questo sovrascrive.

Metodo 2 — Eclipse Run Configuration

Run Configurations → *Java Application* → *New*, poi:

Tab	Cosa mettere
Main	Project: HelloWorldSpringBoot, Main class: HelloWorldSpringBootApplication
Arguments → VM arguments	-Dserver.port=8083
Source	Add → il progetto

Ne crei una per porta e le lanci dalla **Boot Dashboard**, che mostra tutti i microservizi con relativa console.

Questa tecnica viene riusata identica nelle lezioni sul load balancer, dove il docente passa *due* parametri:

```
java -Dserver.instance.id=1 -Dserver.port=8081 -jar ...jar
```

```
java -Dserver.instance.id=2 -Dserver.port=8082 -jar ...jar
```

Il secondo parametro serve a far dire all'istanza *chi è*, così quando testi il bilanciamento vedi da quale istanza arriva la risposta.

Eureka

Definizione. Eureka è un **Service Registry / Discovery Server**: una rubrica dinamica dove ogni microservizio si registra all'avvio dichiarando nome, IP e porta, e dove gli altri microservizi vanno a chiedere "dove sta il servizio X?".

Il problema che risolve, ed è il punto più importante di tutto il corso: **non puoi scolpire IP e porte nel codice**. Se scrivi `http://127.0.0.1:8080` dentro dieci servizi e poi quella porta cambia, devi trovare e modificare tutti e dieci. In un sistema con 40 microservizi e istanze che nascono e muoiono dinamicamente è ingestibile.

Con Eureka nel codice resta **solo il nome logico** del servizio. L'indirizzo lo risolve il registro a runtime.

Come funziona

```
[Service Provider] --(1) register: "mi chiamo PippoService, sono su 10.0.0.5:8080"--> [EUREKA SERVER]

--(2) heartbeat ogni 30s ----->

[Service Consumer] --(3) "dammi l'indirizzo di PippoService" ----->

<--(4) 10.0.0.5:8080 -----

--(5) chiamata diretta --> [Service Provider]
```

Tre operazioni del provider: **register** (all'avvio), **renew** (heartbeat periodico per dire "sono vivo"), **cancel** (alla chiusura). Se gli heartbeat smettono di arrivare, Eureka rimuove l'istanza dal registro.

Il nome "Netflix" nelle dipendenze c'è perché Eureka nasce in Netflix, che ha sviluppato molti dei componenti poi confluiti in Spring Cloud.

L'Eureka Server (HelloWorldSpringCloudEurekaServer2)

Dipendenza: `spring-cloud-starter-netflix-eureka-server`. Poi:

```
@SpringBootApplication
@EnableEurekaServer      // ← una annotazione e il registro è pronto
public class HelloWorldSpringCloudEurekaServer2Application { ... }
```

```
server.port=8761
```

```
eureka.client.register-with-eureka=false
```

```
eureka.client.fetch-registry=false
```

Le due righe sotto sono essenziali e il docente le spiega male nella trascrizione, quindi te le chiarisco:

- **register-with-eureka=false**: il server non deve registrarsi come client di sé stesso.
- **fetch-registry=false**: il server non deve scaricare il registro da un altro Eureka (serve quando hai un cluster di Eureka, qui è singolo).

Senza queste due righe l'applicazione parte comunque ma sporca i log con tentativi di connessione falliti.

8761 è la porta convenzionale. Su `http://localhost:8761` c'è una **dashboard web** che mostra tutte le istanze registrate.

L'Eureka Client (HelloWorldSpringCloudEurekaClient-2)

Dipendenze: `spring-cloud-starter-netflix-eureka-client` + `spring-boot-starter-web`.

```
@SpringBootApplication // ← nessuna annotazione aggiuntiva necessaria!  
  
public class HelloWorldSpringCloudEurekaClient2Application { ... }
```

Attenzione a questo punto: il docente sottolinea che sulla classe principale del client **non serve aggiungere niente**. Dalle versioni recenti di Spring Cloud, `@EnabledDiscoveryClient` è opzionale: la sola presenza della dipendenza sul classpath attiva la registrazione automatica. (Negli altri progetti dello zip la trovi ancora scritta, è retaggio di versioni precedenti e non fa danno.)

Tutto avviene nel properties:

```
spring.application.name=HelloWorldSpringCloudEurekaClient-2  
  
eureka.client.serviceUrl.defaultZone= ${EUREKA_URI:http://localhost:8761/eureka}  
eureka.client.serviceUrl.instance.preferIpAddress= true
```

spring.application.name: il nome con cui il servizio viene registrato. È la chiave di tutto: chi vorrà chiamarlo userà questa stringa.

- **defaultZone:** l'indirizzo di Eureka. La sintassi `${EUREKA_URI:default}` significa "usa la variabile d'ambiente `EUREKA_URI`, se non esiste usa `http://localhost:8761/eureka`". Utile per Docker/Kubernetes.
- **preferIpAddress:** registrati con l'IP invece che con l'hostname (utile in ambienti dove il DNS non risolve gli hostname dei container).

E il servizio esposto:

```
@RestController  
  
@RequestMapping("/api")  
  
public class HelloWorldCtr {
```

```

    @RequestMapping(value = "/ciao", method = RequestMethod.GET, produces =
"application/json")

    public ResponseEntity<String> helloWorld() {

        return new ResponseEntity<String>("helloooo", HttpStatus.OK);

    }

}

```

Avviando il client e ricaricando `localhost:8761` compare la riga con l'istanza registrata.

Esempio Eureka + Feign

Questa lezione **unisce le due precedenti** ed è il pattern che userai davvero in produzione. Tre applicazioni:

```

[Eureka Server 2 :8761]

    ↑ register      ↑ register + discovery
[Client 2 :8080] ←—— [Client 3 :8083]

esponde /api/ciao      esponde /imp/new, chiama Client 2 via Feign

```

Client 3 (HelloWorldSpringCloudEurekaClient-3)

Dipendenze: web + **openfeign** + **eureka-client**.

```

@SpringBootApplication
@EnableFeignClients

public class HelloWorldSpringCloudEurekaClient3Application { ... }

```

L'interfaccia Feign, ed è **qui la differenza cruciale** rispetto alla lezione 5:

```

@FeignClient("HelloWorldSpringCloudEurekaClient-2") // ← SOLO il nome, niente url!

public interface HelloWorldCtrClient {

    @RequestMapping(value = "/api/ciao", method = RequestMethod.GET, produces =
"application/json")

    public ResponseEntity<String> helloWorld();

}

```

Confronta con la lezione 5:

```

// PRIMA (senza Eureka): indirizzo cablato

@FeignClient(name = "HelloWorldSpringBoot", url = "localhost:8080")

```

```
// ORA (con Eureka): solo il nome logico
```

```
@FeignClient("HelloWorldSpringCloudEurekaClient-2")
```

Sparito l'URL. Quel nome **deve essere identico** al `spring.application.name` di Client 2, perché è la chiave con cui Feign interroga Eureka.

Il controller:

```
@RestController
```

```
@RequestMapping("/imp")
```

```
public class HelloCtr {
```

```
    @Autowired
```

```
    private HelloWorldCtrClient helloWorldCtrClient;
```

```
    // imp/new --> api/ciao --> hello
```

```
    @RequestMapping(value = "/new", method = RequestMethod.GET, produces =  
"application/json")
```

```
    public ResponseEntity<String> helloWorld() {
```

```
        String res = helloWorldCtrClient.helloWorld().getBody();
```

```
        return new ResponseEntity<String>(res, HttpStatus.OK);
```

```
    }
```

```
}
```

Sequenza di avvio (l'ordine conta)

1. **Eureka Server** per primo
2. **Client 2**, che si registra
3. **Client 3**, che si registra e interroga il registro

Poi chiami `http://localhost:8083/imp/new` e ottieni "hello".

Cosa è successo dietro le quinte: Feign ha preso il nome

`HelloWorldSpringCloudEurekaClient-2`, ha chiesto a Eureka la lista delle istanze, ne ha scelta una, ha costruito l'URL completo e ha fatto la chiamata HTTP. Tutto automatico. Il docente lo dice bene: "dietro le quinte fa tutto lui da solo".

Introduzione al Bilanciamento Carico

Definizione. Il **load balancer** è il componente che distribuisce le richieste in ingresso su più istanze dello stesso servizio, evitando che una sola si sovraccarichi.

Lo scenario: AddressService2 su 8082 riceve troppe richieste ed è un collo di bottiglia. Avvii una seconda istanza su 8081. Ma ora chi decide, per ogni singola richiesta, quale delle due chiamare? Il load balancer.

Il docente segnala due cose che vale la pena fissare:

1. Il load balancer dialoga con il Service Registry. Non può bilanciare su istanze che non conosce: chiede a Eureka la lista aggiornata delle istanze vive.

2. Esistono due modalità architetturali:

Modalità	Dove sta il balancer	Caratteristiche
Server-side	Componente separato davanti ai servizi	Classico (nginx, HAProxy, un ELB). Singolo punto da gestire, potenziale collo di bottiglia
Client-side	Una libreria dentro ogni client	Nessun hop di rete extra, ogni client sceglie l'istanza più vicina. È quello che usa Spring Cloud

Il docente motiva il client-side così: quando i client sono distribuiti geograficamente in cloud, conviene che ognuno scelga un'istanza vicina alla propria zona.

Spring Cloud LoadBalancer è client-side. Questo è il punto da tenere a mente per la lezione successiva.

Spring Cloud Load Balancer – Parte Uno

Configurazione **manuale**, senza Eureka: serve a capire il meccanismo interno prima di lasciarlo fare a Spring in automatico.

Il servizio (HelloWorldSpringCloudLoadBalancerService_1)

```
@RestController
@RequestMapping("/api")
public class HelloCtr {

    @Value("${server.instance.id}") // ← passato da riga di comando
    String instanceId;

    @RequestMapping(value = "/ciao", method = RequestMethod.GET, produces =
"application/json")
    public String helloWorld() {
        //java -Dserver.instance.id=1 -Dserver.port=8081 -jar ...jar
        //java -Dserver.instance.id=2 -Dserver.port=8082 -jar ...jar
        System.out.println("instanceId:" + instanceId);
        String tmp = "ciao da " + instanceId;
```

```

    return tmp;
}
}

```

Il trucco didattico: la risposta contiene l'ID dell'istanza, così vedi a occhio nudo il bilanciamento. Avvii due istanze con i comandi nei commenti e ottieni "ciao da 1" su 8081 e "ciao da 2" su 8082.

Il client (HelloWorldSpringCloudLoadBalancerClient_1)

Dipendenze: spring-cloud-starter-loadbalancer + spring-boot-starter-webflux (serve WebClient).

Servono **tre classi**, e capirle è utile perché mostrano cosa fa Spring automaticamente quando c'è Eureka.

Classe 1 — il fornitore della lista di istanze:

```

public class HelloWorldSpringBootInstanceSupplier implements ServiceInstanceListSupplier {
    private final String serviceId;

    public HelloWorldSpringBootInstanceSupplier(String serviceId) {
        this.serviceId = serviceId;
    }

    @Override
    public String getServiceId() { return serviceId; }

    @Override
    public Flux<List<ServiceInstance>> get() {
        return Flux.just(Arrays.asList(
            new DefaultServiceInstance(serviceId + "1", serviceId, "localhost", 8082, false),
            new DefaultServiceInstance(serviceId + "2", serviceId, "localhost", 8081, false)));
    }
}

```

Questa è **la lista delle istanze scritta a mano**. Il docente è esplicito sul fatto che è didattica: "teoricamente questa classe dovrebbe essere sostituita con un Eureka server". Con Eureka, questo bean è fornito automaticamente e la lista è dinamica.

Classe 2 — registrare il supplier:

```

@Configuration
public class ServerInstanceConfiguration {

    @Bean
    ServiceInstanceListSupplier serviceInstanceListSupplier() {

        return new
HelloWorldSpringBootTestInstanceSupplier("HelloWorldSpringCloudLoadBalancerService_1");
    }
}

```

Classe 3 — il WebClient bilanciato:

```

@Configuration
@LoadBalancerClient(name = "HelloWorldSpringCloudLoadBalancerService_1",
                    configuration = ServerInstanceConfiguration.class)
public class WebClientConfig {

    @LoadBalanced    // ← l'annotazione che rende il client "consapevole" del balancer
    @Bean
    WebClient.Builder webClientBuilder() {

        return WebClient.builder();
    }
}

```

@LoadBalanced è l'annotazione chiave: intercetta il `WebClient` e, quando l'URL contiene un nome di servizio invece di un host reale, lo risolve passando dal load balancer.

Il main:

```

ConfigurableApplicationContext ctx = new SpringApplicationBuilder(...)
    .web(WebApplicationType.NONE)    // ← non è un server web, solo un client
    .run(args);

WebClient loadBalancedClient = ctx.getBean(WebClient.Builder.class).build();

for(int i = 1; i <= 10; i++) {
    String response = loadBalancedClient.get()
        .uri("http://HelloWorldSpringCloudLoadBalancerService_1/api/ciao")

```

```
.retrieve().toEntity(String.class).block().getBody();

System.out.println(response);
}
```

Nel file dello zip l'URI è rimasto `http://localhost:8081/api/ciao` da un test, ma quello **corretto** per far funzionare il bilanciamento è il **nome del servizio**, come spiega il docente nel video ("gli dico: invoca l'applicazione con questo nome"). Il ciclo di 10 chiamate stampa alternativamente "ciao da 1" e "ciao da 2".

Spring Cloud Load Balancer -Parte Due

Stessa cosa, ma nella **configurazione realistica**: LoadBalancer + Eureka + Feign. E la differenza rispetto alla parte uno è che qui **non scrivi nessuna delle tre classi di prima**.

Il servizio (HelloWorldSpringCloudLoadBalancerService2)

Identico a prima, ma con Eureka:

```
spring.application.name=HelloWorldSpringCloudLoadBalancerService2

eureka.client.serviceUrl.defaultZone= ${EUREKA_URI:http://localhost:8761/eureka}
eureka.client.serviceUrl.instance.preferIpAddress= true
```

Sulla dashboard Eureka compare **un solo servizio con due istanze**.

Il client (HelloWorldSpringCloudLoadBalancerClient2)

```
@SpringBootApplication
@EnableFeignClients
@EnableDiscoveryClient
public class HelloWorldSpringCloudLoadBalancerClient2Application { ... }
```

```
@FeignClient(name = "HelloWorldSpringCloudLoadBalancerService2")
public interface HelloWorldSpringCloudLoadBalancerService2 {

    @RequestMapping(value = "/api/ciao", method = RequestMethod.GET, produces =
"application/json")
    public String helloWorld();
}
```

```

@RestController
public class CustomerService {

    @Autowired
    HelloWorldSpringCloudLoadBalancerService2 helloWorldSpringCloudLoadBalancerService_2;

    @RequestMapping(value = "/getName", method = RequestMethod.GET, produces =
"application/json")
    public String helloWorld() {
        String res = helloWorldSpringCloudLoadBalancerService_2.helloWorld();
        System.out.println("res:" + res);
        String tmp = "abbiamo chiamato " + res;
        return tmp;
    }
}

```

Il risultato: chiami ripetutamente `http://localhost:8084/getName` e ottieni
alternativamente:

```

abbiamo chiamato ciao da 1
abbiamo chiamato ciao da 2
abbiamo chiamato ciao da 1
...

```

Round robin, gratis, senza aver scritto una riga di codice di bilanciamento.

Due note importanti

Sulla dipendenza: il docente nota che `spring-cloud-starter-loadbalancer` è già inclusa transitivamente in `spring-cloud-starter-netflix-eureka-client`, quindi dichiararla esplicitamente è ridondante (nel progetto c'è comunque, non fa danno). Se invece usi `LoadBalancer` **senza** Eureka, la devi dichiarare tu, come nella Parte Uno.

Una precisazione tecnica. Nel video il docente dice "è direttamente Eureka Server che funge da load balance". Non è esatto, e vale la pena chiarirlo perché è concettualmente importante: **Eureka fornisce solo la lista delle istanze**, la scelta di quale chiamare la fa **Spring Cloud LoadBalancer che gira dentro il client**. È bilanciamento *client-side*, esattamente come descritto nella lezione 9. Eureka non vede mai passare il traffico applicativo.

Ultimo dettaglio: questo esempio usa **Spring Cloud LoadBalancer**, non Netflix Ribbon. Ribbon era il vecchio balancer di Spring Cloud, ora in manutenzione e sostituito. Se trovi tutorial con Ribbon, sono obsoleti.

Spring Cloud Gateway - Introduzione
Definizione. Spring Cloud Gateway è un API Gateway: un unico punto di ingresso posto davanti a tutta l'architettura a microservizi. Ogni richiesta esterna passa da lì e viene instradata verso il microservizio giusto.

Perché serve un punto d'ingresso unico. Il docente elenca:

- **autenticazione** centralizzata (verifichi una volta sola chi sta chiamando)
- **autorizzazione** (questo utente può accedere a questo servizio?)
- **logging** centralizzato
- in generale, tutte le funzionalità **trasversali** a tutti i microservizi

Il vantaggio pratico: il client esterno conosce **un solo indirizzo**. I microservizi dietro possono cambiare porta, moltiplicarsi, spostarsi, senza che nessuno all'esterno se ne accorga.

Il progetto (HelloWorldSpringCloudGatewayApi)

Dipendenze: spring-cloud-starter-gateway + spring-cloud-starter-netflix-eureka-client.

Classe principale: **niente annotazioni aggiuntive.**

```
spring.application.name=api-gateway
```

```
server.port=8765
```

```
eureka.client.serviceUrl.defaultZone= http://localhost:8761/eureka
```

```
spring.cloud.gateway.discovery.locator.enabled=true
```

La riga magica è l'ultima. discovery.locator.enabled=true attiva il **DiscoveryClient Route Definition Locator**: il Gateway legge il registro Eureka e crea **automaticamente una route per ogni servizio registrato**, con il pattern:

```
http://gateway:8765/{NOME-SERVIZIO}/{path-originale}
```

Zero configurazione di routing. Se registri un nuovo microservizio su Eureka, il Gateway lo espone da solo.

Il test

Prima, senza Gateway:

```
http://localhost:8080/api/ciao → "hello"
```

Dopo, tramite Gateway:

```
http://localhost:8765/HELLOWORLDSRINGCLOUDEUREKACLIENT-2/api/ciao → "hello"
```

Nota che il nome del servizio va scritto **in maiuscolo**: Eureka normalizza in maiuscolo i nomi registrati.

Il flusso: richiesta al Gateway → il Gateway chiede a Eureka l'indirizzo di quel servizio → Eureka risponde `localhost:8080` → il Gateway inoltra la richiesta e restituisce la risposta.

Spring Cloude Gateway – Route

Definizione. Una **route** è la regola che dice al Gateway "quando arriva una richiesta che soddisfa questo *predicato*, applica questi *filtri* e inoltrala a questo *URI*".

Tre componenti, da memorizzare:

Componente	Cosa fa	Esempio
Predicate	condizione di match	<code>path("/get")</code>
Filter	modifica request/response in transito	<code>addRequestHeader(...)</code>
URI	destinazione	<code>http://httpbin.org:80</code> oppure <code>lb://NOME-SERVIZIO</code>

Serve quando il routing automatico della lezione precedente non basta: vuoi URL personalizzati, vuoi aggiungere header, vuoi instradare verso servizi esterni.

Configurazione via Java (`ApiGatewayConfiguration`)

`@Configuration`

```
public class ApiGatewayConfiguration {
```

`@Bean`

```
public RouteLocator customRouteLocator(RouteLocatorBuilder builder) {
```

```
    return builder.routes()
```

```
        .route(p -> p.path("/get"))
```

```
            .filters(h -> h.addRequestHeader("MioHeader", "ProvaHeader"))
```

```
                .addRequestParameter("MioParam", "ProvaParametro"))
```

```
            .uri("http://httpbin.org:80"))
```

```
        .route(p ->
```

```
p.path("/api/**").uri("lb://HELLOWORLDSRINGCLOUDEUREKACLIENT-2"))
```

```
        .build();
```

```
    }
```

```
}
```

Analizziamo le due route.

Route 1 — verso un sito esterno con filtri.

- `p.path("/get")`: predicato, matcha le richieste su `/get`

- `.filters(...)`: aggiunge un **header** `MioHeader`: `ProvaHeader` e un **query parameter** `MioParam=ProvaParametro` alla richiesta in uscita
- `.uri("http://httpbin.org:80")`: destinazione (`httpbin.org` è un servizio di echo, ti rimanda indietro in JSON tutto quello che gli mandi, comodo per verificare che i filtri abbiano funzionato)

Chiamando `http://localhost:8765/get` la risposta JSON di `httpbin` mostra header e parametro aggiunti dal Gateway. Il docente sottolinea che **questo è il meccanismo su cui si costruisce la sicurezza**: il Gateway autentica l'utente e inietta un header (tipicamente un token JWT) che i microservizi a valle leggono.

Route 2 — verso un microservizio via Eureka.

```
.route(p -> p.path("/api/**").uri("lb://HELLOWORLDSRINGCLOUDEUREKACLIENT-2"))
```

- `"/api/**"`: il doppio asterisco matcha qualsiasi cosa sotto `/api/`, quindi `/api/ciao`, `/api/x/y`, tutto
- `lb://` è lo schema fondamentale: significa **"load balanced"**. Non è un URL vero: dice al Gateway di chiedere a Eureka le istanze registrate con quel nome e di bilanciare tra loro

Il docente lo spiega con precisione: IP e porta "non li posso scolpire nel codice perché queste informazioni sono gestite dal server [Eureka]". `lb://` è esattamente il modo di dire "risolvilo tu".

Risultato: `http://localhost:8765/api/ciao` → `"hello"`.

La stessa configurazione via properties

```
spring.cloud.gateway.routes[0].uri=lb://HELLOWORLDSRINGCLOUDEUREKACLIENT-2
spring.cloud.gateway.routes[0].predicates[0]=Path=/api/**
```

Perfettamente equivalente alla route 2 in Java. Il docente lo dice esplicitamente: "questa configurazione tramite file di property è identica alla configurazione che abbiamo fatto tramite Java".

Quando usare l'una o l'altra: le properties sono più leggibili e modificabili senza ricompilare (e si sposano con Config Server); Java serve quando ti serve logica condizionale nella definizione delle route.

Sulle lambda: il docente mostra prima la versione con la funzione estratta in una variabile `Function<PredicateSpec, Buildable<Route>> f = ...` (la trovi commentata nel file) per chi non ha dimestichezza con le lambda, poi dice che la forma corretta è scriverla inline dentro `.route(...)`.

Spring Cloud Gateway – Filtro di Logging

Definizione. Un **GlobalFilter** è un filtro che il Gateway applica a **tutte** le richieste in transito, indipendentemente dalla route. Il classico caso trasversale: logging, auditing, controlli di sicurezza, metriche.

Distinzione utile:

- **GatewayFilter**: applicato a una route specifica (come `addRequestHeader` della lezione precedente)
- **GlobalFilter**: applicato a tutte le route

Il codice (`LoggingFilter`)

```
@Component
public class LoggingFilter implements GlobalFilter {

    private Logger log = LoggerFactory.getLogger(LoggingFilter.class);

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {

        log.info("Path In Ingresso:" + exchange.getRequest().getPath());

        return chain.filter(exchange);
    }
}
```

Tre elementi da capire:

1. **@Component**. Non serve registrare nulla altrove: Spring rileva il bean e il Gateway lo aggiunge automaticamente alla catena.
2. **ServerWebExchange**. È il contenitore che incapsula **request e response** della richiesta corrente. Da lì accedi a tutto: `exchange.getRequest().getPath()`, gli header, il metodo, i cookie.
3. **return chain.filter(exchange)**. Questa riga è **obbligatoria** ed è il punto più delicato: passa il controllo al filtro successivo della catena. Se la ometti, la richiesta **si ferma qui** e non arriva mai al microservizio. (Questo è anche il modo in cui implementi un filtro di autenticazione: se il token non è valido, *non* chiami `chain.filter()` e ritorni invece una risposta 401.)

`Mono<Void>` è un tipo **reactive** (Project Reactor): Spring Cloud Gateway è costruito su WebFlux, non su Servlet, quindi è completamente non bloccante.

Il test

Chiami `http://localhost:8765/api/ciao`, il microservizio risponde "hello", e nella console del Gateway compare:

```
Path In Ingresso:/api/ciao
```

Resilience4j – Introduzione (Retry)

Definizione. **Resilience4j** è una libreria di *fault tolerance* per applicazioni distribuite. Fornisce una serie di pattern per far sì che il guasto di un microservizio non si propaghi all'intero sistema.

Il concetto di **resilienza**: il sistema deve continuare a funzionare (magari in modalità degradata) anche quando una sua parte è rotta.

I pattern che il corso copre:

Pattern	Annotazione	Cosa fa
Retry	@Retry	riprova la chiamata fallita N volte
Circuit Breaker	@CircuitBreaker	smette di chiamare un servizio notoriamente guasto
Rate Limiter	@RateLimiter	limita le richieste per unità di tempo
Bulkhead	@Bulkhead	limita le chiamate concorrenti

Le dipendenze

Servono due dipendenze, e la seconda è quella che tutti dimenticano:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>    <!-- ← indispensabile -->
</dependency>
```

Perché serve AOP. Resilience4j funziona tramite **Aspect Oriented Programming**: le annotazioni come @Retry non fanno nulla da sole, sono intercettate da un *aspect* che avvolge il metodo e ci mette intorno la logica di retry. Senza `spring-boot-starter-aop` le annotazioni vengono ignorate silenziosamente e non capisci perché non funziona nulla.

Il codice (HelloWorldSpringCloudResilience4j)

```
@RestController
public class CircuitBreakerController {
    private Logger log = LoggerFactory.getLogger(CircuitBreakerController.class);

    @GetMapping("esempio")
    // @Retry(name = "default")
```

```
// @Retry(name = "esempio")

@Retry(name = "esempio", fallbackMethod="metodoFallBack")
public String esempio() {
    log.info("dentro servizio");

    ResponseEntity<String> res = new
RestTemplate().getForEntity("http://localhost:8080/api/ciao", String.class);

    return res.getBody();
}

public String metodoFallBack(Exception ex) {
    return "invocazione al metodo fallita";
}
}
```

Le righe commentate mostrano la progressione didattica del video. Seguiamola.

Step 1 — comportamento di default

Con `@Retry(name = "default")`, senza nessuna configurazione: il servizio chiamato è spento, e nel log vedi **"dentro servizio" stampato 3 volte**. Resilience4j ha riprovato 3 volte (default), poi ha propagato l'eccezione al client, che vede una brutta pagina di errore.

Step 2 — configurazione personalizzata

```
spring.application.name=HelloWorldSpringCloudResilience4j
server.port = 8081

resilience4j.retry.instances.esempio.maxAttempts=5
resilience4j.retry.instances.esempio.waitDuration: 1s
#resilience4j.retry.instances.esempio.enableExponentialBackoff=true
```

Il collegamento chiave è il nome dell'istanza. In

`resilience4j.retry.instances.**esempio**`. * la parola `esempio` deve corrispondere esattamente al `name` dell'annotazione `@Retry(name = "esempio")`. Così configuri comportamenti diversi per metodi diversi. Il nome `default` vale per tutti quelli senza configurazione specifica.

Ora nel log: **5 tentativi, distanziati esattamente di 1 secondo.**

Step 3 — exponential backoff

Decommentando `enableExponentialBackoff=true`, i tentativi non sono più equidistanti ma **raddoppiano progressivamente**. Il docente mostra i timestamp del log: 33, 34, 35, 37, 41 secondi, cioè attese di 1, 1, 2, 4 secondi.

Perché è importante: se un servizio è giù perché sovraccarico, martellarlo con retry a intervallo fisso peggiora le cose. L'exponential backoff dà al servizio il tempo di riprendersi. È la strategia consigliata in produzione.

Step 4 — fallback method

```
@Retry(name = "esempio", fallbackMethod="metodoFallBack")

public String esempio() { ... }

public String metodoFallBack(Exception ex) {
    return "invocazione al metodo fallita";
}
```

Invece dello stack trace, il client riceve un messaggio pulito. **Il vincolo da ricordare**, che il docente sottolinea: il metodo di fallback deve avere la **stessa firma** del metodo originale **più un parametro `Exception`** in coda. Se sbagli la firma, Spring non lo trova e ottieni un errore a runtime.

Resilience4J – Circuit Breaker

Definizione. Il **Circuit Breaker** è un pattern che, per analogia con l'interruttore elettrico, **interrompe le chiamate verso un servizio che si è dimostrato guasto**, evitando di sprecare risorse e tempo in tentativi destinati a fallire.

La differenza rispetto al Retry è concettuale e importante:

- **Retry** = "riprovo, magari era un problema momentaneo". Ma se il servizio è giù da un'ora, riprovare 5 volte a ogni richiesta è solo spreco.
- **Circuit Breaker** = "so già che è giù, non provo nemmeno, vado dritto al fallback e risparmio tempo e risorse".

La macchina a stati

È il cuore della lezione:

Stato	Comportamento	Transizione
CLOSED (chiuso)	Circuito chiuso, corrente passa. Le chiamate vengono eseguite normalmente . Il breaker conta i fallimenti	Superata la % di fallimenti configurata → OPEN
OPEN (aperto)	Circuito aperto, corrente non passa. Le chiamate non vengono nemmeno tentate : va direttamente al fallback	Dopo il tempo di attesa configurato → HALF_OPEN
HALF_OPEN	Lascia passare un numero limitato di	Se hanno successo →

Stato	Comportamento	Transizione
(mezzo aperto)	chiamate di prova	CLOSED . Se falliscono → OPEN
Da tenere presente per non confondersi: in elettronica <i>closed</i> = funziona, <i>open</i> = interrotto. Qui è lo stesso, ma è controintuitivo rispetto al linguaggio comune.		
Il ruolo di HALF_OPEN: è il meccanismo di auto-guarigione . Ogni tanto il sistema "tasta il terreno" per vedere se il servizio è tornato su, senza dover riavviare nulla.		
Il codice (HelloWorldSpringCloudResilience4j -1)		

@RestController

```
public class CircuitBreakerController {

    private Logger log = LoggerFactory.getLogger(CircuitBreakerController.class);

    //for /l %g in () do @(curl http://localhost:8080/esempio & timeout /t 5)

    @GetMapping("esempio")
    @CircuitBreaker(name = "default", fallbackMethod = "metodoFallBack")
    public String esempio() {
        log.info("dentro servizio");
        ResponseEntity<String> res = new
RestTemplate().getForEntity("http://localhost:8080/api/ciao", String.class);
        return res.getBody();
    }

    public String metodoFallBack(Exception ex) {
        log.info("dentro metodoFallBack");
        return "tutti i tentativi sono falliti";
    }
}
```

Le due `log.info` sono lo strumento diagnostico e vanno lette insieme:

Cosa vedi nel log	Stato del circuito
dentro servizio + dentro metodoFallBack	CLOSED : prova davvero, fallisce, ripiega
solo dentro metodoFallBack	OPEN : non prova nemmeno
ricompare dentro servizio ogni tanto	HALF_OPEN : sta testando

management.health.circuitbreakers.enabled=**true**

```
management.endpoint.health.show-details=always
management.endpoints.web.exposure.include=health
```

Queste tre righe espongono lo stato dei circuit breaker su `/actuator/health`. Utile per il monitoraggio in produzione (e richiedono `spring-boot-starter-actuator`, presente nel pom).

Come generare il carico per vedere le transizioni

Con una chiamata sola non vedi niente: il breaker si apre solo dopo un certo numero di fallimenti. Il docente mostra due metodi.

Postman Collection Runner: crei una collection con dentro la richiesta, poi *Run* e imposti il numero di iterazioni (1000). Attenzione al problema che gli capita nel video: se non fai **Save** della request dentro la collection, il Runner la esegue con URL vuoto.

Riga di comando Windows (nel commento del codice):

```
for /l %g in () do @(curl http://localhost:8080/esempio & timeout /t 5)
```

Ciclo infinito che chiama l'endpoint ogni 5 secondi. Su Linux si fa con `watch curl ...` o un `while true` con `sleep`.

Cosa si osserva

Lanciando 1000 richieste sul servizio guasto, nel log si vede la sequenza completa: prima entrambe le log (CLOSED), poi solo il fallback (OPEN), poi ricompare sporadicamente "dentro servizio" (HALF_OPEN), poi di nuovo solo fallback (di nuovo OPEN, perché il servizio è ancora giù). Se riavviassi il servizio chiamato, dal HALF_OPEN successivo il circuito tornerebbe CLOSED da solo.

Resilience4J – Configurazione (Rate Limiter e Bulkhead)

Ultimi due pattern, entrambi in `HelloWorldSpringCloudResilience4j-2`.

```
@RestController
```

```
public class CircuitBreakerController {
```

```
    private Logger log = LoggerFactory.getLogger(CircuitBreakerController.class);
```

```
    @GetMapping("esempio")
```

```
    // @RateLimiter(name="default") //10 sec per 10000 chiamate
```

```
    @Bulkhead(name="default")
```

```
    public String esempio() {
```

```
        log.info("dentro servizio");
```

```
        return "OKKKKKK";
```

```
}  
}
```

2 req ogni 5 sec

```
resilience4j.ratelimiter.instances.default.limitForPeriod  
=2 resilience4j.ratelimiter.instances.default.limitRefreshPeriod=5s resilience4j.ratelimiter.instances.  
default.timeOutDuration=0  
resilience4j.bulkhead.instances.default.MaxConcurrentCalls=2
```

Rate Limiter

Definizione. Limita il **numero di chiamate accettate in una finestra temporale**. Serve a proteggere un servizio dal sovraccarico e a implementare politiche di throttling (es. piani API a consumo).

Property	Significato
limitForPeriod	quante chiamate accettare per periodo (qui 2)
limitRefreshPeriod	durata del periodo (qui 5 secondi)
timeOutDuration	quanto una chiamata in eccesso attende prima di arrendersi

Il comportamento cambia radicalmente in base a `timeOutDuration`, ed è la parte più istruttiva della lezione:

Senza `timeOutDuration` (default diverso da zero): lanciando 10 richieste, ne passano 2, poi c'è una pausa, poi altre 2, e così via. **Tutte e 10 ricevono HTTP 200**, semplicemente sono rallentate. Le richieste in eccesso *si mettono in coda* aspettando il rinnovo del permesso.

Con `timeOutDuration=0`: le richieste in eccesso non aspettano affatto e **falliscono immediatamente**. Nel test del docente le prime 2 passano, le successive vanno tutte in errore, e l'esecuzione è molto più rapida.

La scelta dipende dal caso d'uso: attesa se vuoi solo livellare i picchi senza perdere richieste; errore immediato se vuoi fail-fast e lasciare al chiamante la decisione su cosa fare.

Il valore di default, se non configuri niente, è **10.000 chiamate ogni 10 secondi** (praticamente illimitato).

Bulkhead

Definizione. Limita il numero di **chiamate concorrenti simultanee** a un metodo.

Il nome viene dalla nautica: le *paratie stagne* di una nave. Se uno scompartimento si allaga, le paratie impediscono all'acqua di invadere tutta la nave. Allo stesso modo, se un servizio lento comincia a occupare tutti i thread disponibili, il bulkhead limita quanti thread può monopolizzare, lasciando risorse per le altre funzionalità dell'applicazione.

```
resilience4j.bulkhead.instances.default.MaxConcurrentCalls=2
```

Massimo 2 chiamate contemporanee a quel metodo. La terza viene rifiutata (o attende, a seconda della configurazione).

Rate Limiter e Bulkhead misurano cose diverse, non confonderli:

	Misura	Domanda a cui risponde
Rate Limiter	chiamate nel tempo	"quante richieste al secondo accetto?"
Bulkhead	chiamate contemporanee	"quante richieste posso avere in corso nello stesso istante?"

Il quadro d'insieme

Ricomponendo tutte le lezioni, ecco l'architettura completa che il corso costruisce pezzo per pezzo



E il riepilogo delle annotazioni, che è quello che ti serve avere in testa:

Annotazione	Dove va	Cosa fa
@EnableConfigServer	main del Config Server	attiva il server di configurazione
@EnableEurekaServer	main dell'Eureka Server	attiva il registro
@EnableFeignClients	main del chiamante	fa scansionare le interfacce @FeignClient
@FeignClient(name=...)	interfaccia	client HTTP dichiarativo
@LoadBalanced	bean	abilita il bilanciamento client-side

Annotazione	Dove va	Cosa fa
	WebClient/RestTemplate	
@Retry	metodo	riprova su fallimento
@CircuitBreaker	metodo	interrompe le chiamate a servizi guasti
@RateLimiter	metodo	limita richieste/tempo
@Bulkhead	metodo	limita chiamate concorrenti
Le porte convenzionali da ricordare: 8888 Config Server, 8761 Eureka, 8765 Gateway (quest'ultima è la scelta del docente, non uno standard).		